UNITÀ 2 · PROGETTARE E UTILIZZARE WEB SERVICE

Simulazione della verifica dell'Unità 2

Stessa struttura, stessa difficoltà e stessi argomenti della verifica vera, con domande diverse. Prova a farla in 90 minuti senza appunti, poi correggiti con le soluzioni in fondo. Punteggio totale: 100 punti.

Nome e cognome | Classe | Data

Il caso. La scuola vuole un'API, **EventiAPI** (inventata), per gli eventi del pomeriggio: conferenze, hackathon, open day. Gli studenti si iscrivono dall'app o dal sito; ogni evento ha un numero massimo di posti. Gli esercizi 3, 5, 6, 7, 8 e 9 si riferiscono a questo caso.

1. Scelta multipla

16 punti · 2 per domanda

Segna una sola risposta per ogni domanda.

a) Quale di queste è una delle "fallacie del calcolo distribuito"?
- ○ I server hanno sempre più RAM dei client
- ○ La rete è affidabile
- ○ I database non si guastano mai
- ○ HTTP è senza stato

b) La semantica di invocazione "al più una volta" si ottiene se…
- ○ il client ritrasmette finché riceve risposta
- ○ il client non ritrasmette mai e non aspetta
- ○ il server ricorda gli ID delle richieste e scarta i duplicati
- ○ si usa sempre UDP

c) Quale compito NON è tipico di un API gateway?
- ○ instradare /ordini al servizio Ordini
- ○ verificare chiavi e token
- ○ limitare il numero di richieste
- ○ eseguire le query sul database di ogni servizio

d) Quale frammento XML è ben formato?
- ○ `<a><b></a></b>`
- ○ `<voto valore=9/>`
- ○ `<a><b/></a>`
- ○ `<Nome>Ada</nome>`

e) Un messaggio SOAP 1.2 su HTTP ha Content-Type…
- ○ `application/json`
- ○ `application/soap+xml`
- ○ `text/html`
- ○ `multipart/form-data`

f) gRPC usa…
- ○ XML e WSDL su HTTP/1.1
- ○ JSON-RPC su WebSocket
- ○ Protocol Buffers su HTTP/2
- ○ oggetti Java serializzati sulla porta 1099

g) Quale scelta viola il vincolo REST "stateless"?
- ○ Il token viaggia in ogni richiesta
- ○ Il server ricorda in memoria l'ultimo evento visto da ogni utente
- ○ Gli eventi si filtrano con `?data=2026-12-03`
- ○ Le risposte hanno `Cache-Control`

h) Per il browser, `http://localhost:5500` e `http://127.0.0.1:5500` sono…
- ○ la stessa origine, perché sono lo stesso computer
- ○ origini diverse, perché l'host è diverso
- ○ la stessa origine, perché la porta è uguale
- ○ origini diverse, perché lo schema è diverso

2. Vero o falso

10 punti · 1 per frase

- V ☐ F ☐ In un sistema distribuito i nodi comunicano usando una memoria condivisa.
- V ☐ F ☐ Un ESB può trasformare un formato di messaggio in un altro.
- V ☐ F ☐ Secondo la legge di Conway, i sistemi copiano la struttura di comunicazione di chi li progetta.
- V ☐ F ☐ JSON ammette i commenti, come JavaScript.
- V ☐ F ☐ Il codice 304 dice al client che può usare la copia che ha in cache.
- V ☐ F ☐ In SOAP l'elemento Header è obbligatorio.
- V ☐ F ☐ In SOAP su HTTP le operazioni di sola lettura usano il metodo GET.
- V ☐ F ☐ Con `HttpServer` del JDK, `sendResponseHeaders(204, -1)` indica una risposta senza corpo.
- V ☐ F ☐ PATCH è sempre idempotente.
- V ☐ F ☐ Una chiave di un'API di AI scritta nel JavaScript della pagina è al sicuro se il sito usa HTTPS.

3. Leggi lo scambio HTTP

12 punti · 2 per domanda

```
GET /api/eventi?data=2026-12-03 HTTP/1.1
Host: eventi.scuola.it
Accept: application/json
```

```
HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: max-age=300

[{"id":7,"titolo":"Hackathon",
  "data":"2026-12-03","posti":40}]
```

a) Qual è la risorsa, e a che cosa serve la parte dopo il `?` _____

b) Che cosa permette l'header Cache-Control, e per quanto tempo? _____

c) Il metodo usato è sicuro? È idempotente? Perché? _____

d) Che cosa dovrebbe rispondere il server se in quella data non ci sono eventi? _____

e) Se il client chiedesse `Accept: application/xml` e il server sapesse produrre solo JSON, che codice riceverebbe? _____

f) Scrivi le due righe di JavaScript che leggono la risposta e stampano il titolo del primo evento.

_____

4. SOAP e WSDL

10 punti

Il vecchio servizio SOAP 1.1 dei certificati di partecipazione risponde così a una richiesta:

```
HTTP/1.1 500 Internal Server Error
Content-Type: text/xml; charset=utf-8

<S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/">
  <S:Body><S:Fault><faultcode>S:Client</faultcode>
    <faultstring>Codice studente non valido</faultstring></S:Fault></S:Body>
</S:Envelope>
```

a) Che cosa significa il faultcode Client? Di chi è la colpa? (2 punti) _____

b) Da che cosa capisci che è SOAP 1.1 e non 1.2? Indica due indizi. (2 punti) _____

c) Nel WSDL, che cosa descrive il *binding*? (2 punti) _____

d) Con quale strumento si genera in Java il client a partire dal WSDL, e che cosa produce? (2 punti)

_____

e) In un'API REST come comunicheresti lo stesso errore? (2 punti) _____

5. Il codice giusto

12 punti · 2 per caso

Situazione	Codice
a) `GET /api/eventi/99`, ma l'evento 99 non esiste	
b) Uno studente prova a iscriversi a un evento che ha già tutti i posti occupati	
c) `POST /api/eventi/7/iscrizioni` riuscito	
d) Il corpo di un POST è un JSON con una parentesi graffa in meno	
e) Un client invia 100 richieste in un secondo e supera il limite	
f) Il server è fermo per una manutenzione programmata	

6. Trova gli errori nel server Java

10 punti · 2,5 per errore

Questo metodo di EventiAPI, eseguito da un pool di thread, gestisce `POST /api/eventi`. Trova **quattro** errori e scrivi la correzione.

```
1  static int prossimo = 1;
2  static void crea(HttpExchange ex) throws IOException {
3    String corpo = new String(ex.getRequestBody().readAllBytes(), StandardCharsets.UTF_8);
4    Evento e = new Evento(prossimo++, campo(corpo, "titolo"), campo(corpo, "data"));
5    eventi.put(e.id(), e);
6    if (e.titolo() == null) { rispondi(ex, 400, errore("titolo mancante")); return; }
7    rispondi(ex, 200, e.toJson());
8  }
```

7. Che cosa stampa?

8 punti

Il server ha 4 eventi e il CORS è configurato bene.

```
console.log("1");
fetch("https://eventi.scuola.it/api/eventi")
  .then(r => r.json())
  .then(ev => console.log("2", ev.length));
console.log("3");
```

a) Scrivi le righe stampate, nell'ordine giusto, e spiega perché. (4 punti)

b) Che cosa succede se il server non invia l'header Access-Control-Allow-Origin e la pagina è su un'altra origine? (2 punti)

c) Riscrivi la chiamata con async/await, controllando r.ok. (2 punti)

8. Progetta EventiAPI

12 punti

a) Scrivi **6 endpoint** per: elencare gli eventi di una data, vedere un evento, creare un evento, iscrivere uno studente a un evento, annullare un'iscrizione, elencare gli iscritti di un evento. Per ognuno: metodo, URI, codice di successo, un codice d'errore. (9 punti)

Metodo	URI	Successo	Errore possibile

b) Una pagina JSP mostra gli eventi chiamando EventiAPI dal server Tomcat. Disegna (o descrivi) la sequenza dei messaggi tra browser, Tomcat ed EventiAPI e spiega perché in questo caso il CORS non serve. (3 punti)

9. Casi di test

4 punti · 2 per caso

Scrivi due casi di test negativi per l'iscrizione a un evento: richiesta e codice atteso.

10. Domanda aperta

6 punti

Con un esempio tratto da EventiAPI, spiega perché una chiamata in rete non è mai uguale a una chiamata locale. Usa almeno tre di questi concetti: guasto parziale, latenza, timeout, idempotenza, ritrasmissione.

Soluzioni

1. Scelta multipla

a) **La rete è affidabile** · b) **il server ricorda gli ID e scarta i duplicati** (se la richiesta si perde, non viene mai eseguita: "al più" una volta) · c) **eseguire le query sul database di ogni servizio** (ogni servizio possiede i suoi dati) · d) `<a><b/></a>` · e) **application/soap+xml** · f) **Protocol Buffers su HTTP/2** · g) **il server ricorda l'ultimo evento visto** · h) **origini diverse, perché l'host è diverso**.

2. Vero o falso

1 **F** (solo messaggi) · 2 **V** · 3 **V** · 4 **F** · 5 **V** · 6 **F** (obbligatori Envelope e Body) · 7 **F** (sempre POST) · 8 **V** (0 vorrebbe dire chunked) · 9 **F** (non garantito) · 10 **F** (HTTPS protegge il viaggio, ma il codice della pagina lo legge chiunque).

3. Scambio HTTP

a) **La collezione /api/eventi; la query string filtra per data** · b) **Client e proxy possono riusare la risposta per 300 secondi (5 minuti)**: è il vincolo cache · c) **GET è sicuro (non cambia lo stato) e idempotente** · d) **200 con un array vuoto** `[]`, non 404: la collezione esiste · e) **406 Not Acceptable** · f) `const ev = await r.json(); console.log(ev[0].titolo);`

4. SOAP e WSDL

a) **Errore causato dalla richiesta del client** (in SOAP 1.2 si chiama Sender) · b) **namespace schemas.xmlsoap.org/soap/envelope/, Content-Type text/xml, elementi faultcode/faultstring** (due bastano) · c) **Come le operazioni del portType viaggiano: protocollo (SOAP su HTTP) e stile (document/literal)** · d) **wsimport (Jakarta XML Web Services / Metro): genera le classi Java dello stub e dei tipi** · e) **Codice 400 (o 422) con un JSON come** `{"errore":"codice studente non valido"}`.

5. Codici

a) **404** · b) **409** · c) **201** (con Location) · d) **400** · e) **429** · f) **503**.

6. Errori nel server

Riga 1/4: `prossimo++` non è atomico con più thread → `AtomicInteger.getAndIncrement()`. **Righe 5–6**: salva prima di validare → controllare prima e salvare dopo. **Riga 7**: una creazione risponde **201**, non 200. **Riga 7**: manca l'header `Location: /api/eventi/{id}`, da impostare prima di rispondere. Accettato anche: `eventi` deve essere una ConcurrentHashMap; manca il controllo della data.

7. Che cosa stampa

a) **1, 3, poi 2 4**: fetch è asincrona, le funzioni nei then partono quando arriva la risposta, dopo che il codice sincrono è finito. b) **Stampa 1 e 3; il browser blocca la risposta, la Promise viene rifiutata (TypeError "Failed to fetch") e "2" non compare**. c) `const r = await fetch(url); if (!r.ok) throw new Error("HTTP " + r.status); const ev = await r.json(); console.log("2", ev.length)` dentro una funzione async.

8. Progetto

Metodo e URI	Successo	Errore
`GET /api/eventi?data=2026-12-03`	200	400 data non valida
`GET /api/eventi/{id}`	200	404
`POST /api/eventi`	201 + Location	400, 401, 403
`POST /api/eventi/{id}/iscrizioni`	201	404, 409 posti esauriti o già iscritto
`DELETE /api/eventi/{id}/iscrizioni/{idIscrizione}`	204	404
`GET /api/eventi/{id}/iscrizioni`	200	404, 403

b) **Browser → GET pagina.jsp a Tomcat → Tomcat (servlet generata) → GET /api/eventi a EventiAPI → JSON → Tomcat scrive l'HTML con l'escape → HTML al browser.** Il CORS non serve perché la chiamata all'API la fa il server Tomcat, non uno script nel browser: il CORS è una regola dei browser.

9. Casi di test

Esempi: `POST /api/eventi/9999/iscrizioni` **→ 404**; **iscrizione a evento pieno → 409**; **corpo senza codice studente → 400**; **senza token → 401**; **stesso studente iscritto due volte → 409**.

10. Domanda aperta

Punti attesi: l'app chiama `POST /iscrizioni` e non riceve risposta entro il **timeout**; non sa se il server ha iscritto lo studente (**guasto parziale**: rete, server o risposta persa). Se **ritrasmette**, un POST non **idempotente** può creare due iscrizioni e occupare due posti: serve un controllo dei duplicati (409) o una chiave di idempotenza. Anche senza guasti c'è la **latenza**: una chiamata in rete dura millisecondi, una locale nanosecondi. 2 punti per concetto usato correttamente, fino a 6.